

Database Migration Strategy — DocumentIA

1. OVERVIEW

Using: **Entity Framework Core 8** with SQL Server

Migrations are stored in: `src/backend/DocumentIA.Data/Migrations/` (project `DocumentIA.Data`, startup project `DocumentIA.Functions`)

Current schema version: **v1.5.0** (19 migrations total)

2. LOCAL DEVELOPMENT MIGRATIONS

Creating a New Migration

```
# From the repository root. Migrations live in DocumentIA.Data;
# DocumentIA.Functions is the startup project (resolves the DbContext configuration).
dotnet ef migrations add "AddNewColumn_YourFeature" `
  --project src/backend/DocumentIA.Data `
  --startup-project src/backend/DocumentIA.Functions `
  --context DocumentIADbContext
```

Generated files:

- `Migrations/YYYYMMDDHHMMSS_AddNewColumn_YourFeature.cs` (Up/Down methods)
- `Migrations/DocumentIADbContextModelSnapshot.cs` (Current schema state)

Migration Code Structure

```
public partial class AddNewColumn_YourFeature : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name: "NewColumn",
            table: "Documentos",
            type: "nvarchar(max)",
            nullable: true);

        // Add index if needed
        migrationBuilder.CreateIndex(
            name: "IX_Documentos_NewColumn",
            table: "Documentos",
            column: "NewColumn");
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropIndex("IX_Documentos_NewColumn", "Documentos");
        migrationBuilder.DropColumn("NewColumn", "Documentos");
    }
}
```

Best Practices

1. **One logical change per migration** (not multiple unrelated changes)
2. **Always include `Down()` method** (for rollback)
3. **Add indexes for frequently queried columns**
4. **Use NOT NULL only if you have default value for existing rows**

5. Test locally before committing

3. TESTING MIGRATIONS LOCALLY

Step 1: Fresh Local Database

```
# Delete existing local DB and logs
rm C:\temp\MVP\documento-ia-clasificacion-mvp\data\DocumentIA.db -Force -ErrorAction SilentlyContinue

# Create new DB with all migrations
dotnet ef database update --context DocumentIADbContext
```

Step 2: Verify New Column

```
-- Check if column exists
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Documentos' AND COLUMN_NAME = 'NewColumn'
```

Step 3: Test Queries

```
// In test
using var context = new DocumentIADbContext();
var doc = context.Documentos.FirstOrDefault();
Assert.NotNull(doc.NewColumn); // or nullable check
```

Step 4: Rollback Test

```
# Get list of migrations
dotnet ef migrations list

# Rollback to previous version
dotnet ef database update <previous-migration-name>

# Verify column is gone
dotnet ef database update <new-migration-name> # Apply again
```

4. STAGING DEPLOYMENT (Pre-Prod Validation)

Pre-Migration Checklist

- ☐ Migration tested locally
- ☐ Data script prepared (if data changes needed)
- ☐ Rollback procedure documented
- ☐ Estimated time to completion (< 5 min for < 1M rows)
- ☐ Backup captured
- ☐ No breaking changes to application code

Migration Execution

1. Backup Staging Database:

```
$db = Get-AzSqlDatabase -ResourceGroupName "RG-Staging" -ServerName "doc-ia-sql-staging" -DatabaseName "DocumentIA"
New-AzSqlDatabaseBackup -Database $db -BackupName "pre-migration-backup-$(Get-Date -Format 'yyyyMMdd-HH:mm')"
```

2. Apply the Migration:

```
# The RunMigrations stage in azure-pipelines.yml is disabled by default
# (condition: false). Migrations are applied manually/on-demand by enabling
# that stage or running the EF Core command against the target database:
dotnet ef database update `
  --project src/backend/DocumentIA.Data `
  --startup-project src/backend/DocumentIA.Functions `
  --connection "<SQL_CONNECTION_STRING>"
```

3. Verify Migration Applied:

```
-- Check migration history
SELECT MigrationId, ProductVersion FROM __EFMigrationsHistory
ORDER BY MigrationId DESC LIMIT 1
```

4. Smoke Test:

```
# Run smoke tests against staging
./scripts/testing/smoke-test-release.ps1
```

5. Monitor 1 Hour:

- Check error rate (should be < 1%)
- Check latency (should be < 30 sec P99)
- Monitor memory usage (no spike expected)

If Issues Found

1. Immediate Rollback:

```
# Revert code
git revert <migration-commit>
git push origin develop
# Redeploy (auto-runs Down migration)
```

2. Restore Database:

```
Restore-AzSqlDatabase -ServerName "doc-ia-sql-staging" `
  -DatabaseName "DocumentIA" -BackupName "pre-migration-backup-XXX"
```

3. Post-mortem:

- Document issue
- Fix migration code
- Retry

5. PRODUCTION DEPLOYMENT (0-Downtime Strategy)

Highly Available Deployments

Goal: Apply migration without stopping the application

Method 1: Blue-Green Deployment (Recommended)

1. **Prepare** (before release):

- Spin up replica database (copy from prod backup)
- Run migration on replica
- Verify schema matches expected
- Keep replica ready

2. **Execute** (during release):

- Failover application to replica database
- Application keeps running (connection string updated)
- Update primary database with migration
- Failback after verification

3. **Verify**:

- Monitor error rate (expect brief spike during failover)
- Verify data consistency
- Keep replica for 24 hours as safety net

Method 2: Online Index Rebuild (For Large Tables)

For adding indexed columns on large tables:

```
-- Online (application keeps running)
CREATE NONCLUSTERED INDEX IX_Documentos_NewColumn
ON dbo.Documentos(NewColumn) WITH (ONLINE=ON)
```

Pre-Production Migration Checklist

- ☐ Full backup captured
- ☐ Blue-green environment ready
- ☐ Rollback procedure tested
- ☐ Maintenance window scheduled (window < 5 min)
- ☐ All instances updated to new code
- ☐ Monitoring alerts active

Production Migration Procedure

```
# 1. Backup production database
$db = Get-AzSqlDatabase -ResourceGroupName "RG-Prod" -ServerName "doc-ia-sql" -DatabaseName "DocumentIA"
New-AzSqlDatabaseBackup -Database $db -BackupName "pre-migration-backup-$(Get-Date -Format 'yyyyMMddHHmm')"
```

```
# 2. Deploy new code to production
# Trigger: azure-pipelines-functions.yml stage=production
# Note: Set APPLY_MIGRATIONS=true environment variable
```

```
# 3. Monitor migration progress
# (Check SQL Server logs, migration can take minutes)
```

```
# 4. Verify success
$history = @(Invoke-SqlCmd -ServerInstance "doc-ia-sql.database.windows.net" `
    -Database "DocumentIA" -Query "SELECT TOP 1 * FROM __EFMigrationsHistory ORDER BY MigrationId DESC")
if ($history[0].MigrationId -match "v1.5.0") { Write-Host "Migration applied" }
```

```
# 5. Smoke test
./scripts/testing/smoke-test-release.ps1 -Endpoint "https://documentia-prod.azurewebsites.net"
```

```
# 6. Monitor 1 hour
# - Error rate < 1%
# - P99 latency < 30 sec
# - No data corruption logs
```

6. ROLLBACK PROCEDURES

Automatic Rollback (EF Core)

```
# If migration failed during deployment:
dotnet ef database update <previous-migration-name>
# EF Core runs Down() method to revert schema
```

Manual Rollback (SQL Server)

If automatic rollback fails:

```
-- Get last successful migration
SELECT TOP 1 MigrationId FROM __EFMigrationsHistory ORDER BY MigrationId DESC

-- Delete problematic migration from history
DELETE FROM __EFMigrationsHistory
WHERE MigrationId = '20260610123456_BadMigration'

-- Manually revert schema (if needed)
ALTER TABLE Documentos DROP COLUMN BadColumn
DROP INDEX IX_BadColumn ON Documentos
```

Database Restore (Worst Case)

```
# If data corruption or migration corrupted data:
Restore-AzSqlDatabase -ServerName "doc-ia-sql" `
  -DatabaseName "DocumentIA" -BackupName "pre-migration-backup-XXX"
# Rollback code to previous version
# Redeploy
```

7. MIGRATION MONITORING

During Migration

```
-- Monitor migration progress (if long-running)
SELECT session_id, start_time, status, command, statement_start_offset
FROM sys.dm_exec_requests
WHERE command LIKE 'ALTER%'

-- Estimated time (rough)
-- Rows: 1M → ~1-2 min
-- Rows: 10M → ~5-10 min
-- Rows: 100M+ → Consider maintenance window
```

After Migration

Verify integrity:

```

-- Check all tables accessible
SELECT COUNT(*) as total_tables FROM information_schema.tables WHERE table_schema='dbo'

-- Check for orphaned records (if foreign key added)
SELECT COUNT(*) as orphaned FROM DocumentoEjecuciones d
LEFT JOIN Documentos doc ON d.DocumentoId = doc.DocumentoId
WHERE doc.DocumentoId IS NULL

-- Verify indexes created
SELECT name, type_desc FROM sys.indexes WHERE object_id = OBJECT_ID('dbo.Documentos')

```

8. KNOWN MIGRATION ISSUES & SOLUTIONS

Issue	Cause	Solution
Migration stuck	Long-running operation on large table	Increase timeout, consider maintenance window
Timeout during migration	Network lag or lock contention	Retry, or restore and try again
Data loss after migration	Incorrect Down() method	Restore from backup, fix migration, retry
Connection pool exhausted	Too many concurrent operations	Restart Functions, reduce concurrency